ndarray 的学习

一.解释

ndarray是numpy的数组

二.特性

1.多维性 0/1/2.....维

In [ ]:
```
import numpy as np
```

In [ ]:
```
arr = np.array(5) #创建一个0维数组
print(arr)
print("arr的维度：",arr.ndim) #注意中间用逗号隔开
```

In [ ]:
```
arr = np.array([2, 2, 2]) #创建一个1维数组
#逗号之后加一个空格，输出时不会有逗号,且不受空格数影响
print(arr)
print("arr的维度：",arr.ndim) #注意中间用逗号隔开
```

In [ ]:
```
arr = np.array([[2, 2, 2],[2, 3, 2],[2, 2, 2]]) #创建一个2维数组
#三个中括号外面还有一个中括号
print(arr)
print("arr的维度：",arr.ndim) #注意中间用逗号隔开
```

2.同质性 所有元素类型在创建时ndarray会统一

In [ ]:
```
arr = np.array([1,"luti"])
print(arr)#不同的数据类型会被强制转换为相同数据类型
```

In [ ]:
```
arr = np.array([1,2.3])
print(arr)#输出int转float的时候输出“1.”省略了.后的0
```

3.高效性 连续内存化存储，向量化计算

三.属性

In [ ]:
```
arr = np.array([[1,2,3],[4,5,6],[7,8,9]])
print("数组：",arr)
print("形状：",arr.shape)
print("维度：",arr.ndim)
print("元素类型：",arr.dtype)#是其中元素的类型
"""
int64：固定 8 字节（64 位），位（bit）是最小单位，一字节(byte)等于8bit，64bit就是8字节NumPy 数组省内存——一个 int64 就是实打实的 8 字节，而 Python 的 int 是个对象，随便一个数就得 28 字节起步。
范围 -2^63 ~ 2^63-1,
特点：快、省、稳
"""
print("转置：",arr.T)
#深度学习中会用到，行变列，列变行,对0/1维无用
print("总元素个数：",arr.size)#不是大小
```

In [ ]:
```
arr = np.array(9)
print("数组：",arr)
print("形状：",arr.shape)
print("维度：",arr.ndim)
print("元素类型：",arr.dtype)
print("转置：",arr.T)
print("总元素个数：",arr.size)
```

In [ ]:
```
arr = np.array([2, 3., 2])#注意浮点数写法
print("数组：",arr)
print("形状：",arr.shape)
print("维度：",arr.ndim)
print("元素类型：",arr.dtype)
print("转置：",arr.T)
print("总元素个数：",arr.size)
```

四.创建

1. **基础构造**：适用于手动构建小规模数组或复制已有数据。
2. **预定义形状填充**：用于快速初始化固定形状的数组（如全0占位、全1初始化）。
3. **基于数值范围生成**：生成数值序列，常用于模拟时间序列、坐标网格等。
4. **特殊矩阵生成**：数学运算专用（如线性代数中的单位矩阵）。
5. **随机数组生成**：模拟实验数据、初始化神经网络权重等场景。
6. **高级构造方法**：处理非结构化数据（如文件、字符串）或通过函数生成复杂数组。

用途	方法			
基础构造	np.array()	np.copy()		
预定义形状填充	np.zeros()	np.ones()	np.empty()	np.full()
基于数值范围生成	np.arange()	np.linspace()	np.logspace()	
特殊矩阵生成	np.eye()	np.diag()		
随机数组生成	np.random.rand()	np.random.randn()	np.random.randint()	
高级构造方法	np.array()	np.loadtxt()	np.fromfunction()	

名称	维度	示例	备注
标量	0 维	5, 3.14	单个数字，无行列
向量	1 维	[1, 2, 3]	只有行或列（一维数组）
矩阵	2 维	[[1, 2], [3, 4]]	严格的行列结构（二维表）
张量	≥3 维	[[[1, 2], [3, 4]]]	高阶数组（如 RGB 图像）

矩阵类型	定义	例子
零矩阵	所有元素为 0	[0 0] [0 0]
单位矩阵	对角线上为 1，其余为 0	[1 0] [0 1]
对角矩阵	只有对角线有非零值	[2 0] [0 3]
对称矩阵	$A = A^T$	[1 2] [2 3]

In [ ]:
```
#基础
arr = np.array(0)#有小括号的要行动

list1 = [2 ,2 ,2 ]
arr = np.array(list1,dtype=np.float64)#可以直接改元素类型
```

In [ ]:
```
# copy
arr1 = np.copy(arr)#元素相同但不是一个数组，以下为验证
print(arr1.dtype)
arr2 = np.copy(list1)
print(arr2)#注意这样是可以的
print(arr2.dtype)

print(arr1)
arr1[0] = 4#列表的元素替换
print(arr1)
print(arr)
```

In [ ]:
```
#预定义形状(全零，全一，未初始化，指定数值)
arr1 = np.zeros((3,5),dtype=int)
arr2 = np.ones((3,),dtype=int)#注意这种shape
print(arr1)
print(arr2)
#全零和全一默认是浮点数类型,输入int而不是int64，是因为输入int,解释器会自动匹配适合的位数
arr1 = np.empty((3,5))#只指定形状，内容随机,必须要两个括号
arr2 = np.full((3,5),2025)
print(arr1)
print(arr2)
#高级:
arr1 = np.zeros_like(arr1)#形状像arr，预定义形状都可用
print(arr1)
```

In [ ]:
```
#等差数列
arr = np.arange(1,10,1)
#start起始，end结束，step步长，前闭后开
print (arr)
```

In [ ]:
```
#等间隔数列
arr = np.linspace(1,10,5,dtype=int)
#start起始，end结束，份数，前闭后闭
#份数大于等于2，则1和10必在其中
#默认浮点数
print (arr)
```

In [ ]:
```
#对数间隔数列
arr = np.logspace(0,4,3,base=3)
#先0~4均匀取三个数，再算base的这么多次方；默认浮点数
#base默认是10
print (arr)#算出来的数值是等比，是"在对数尺度上等距"的意思
```

In [ ]:
```
#特殊矩阵:除专门的整数外，都是默认浮点数

#单位矩阵
arr = np.eye(5)
print (arr)
arr = np.eye(3,4)#非方阵，非单位矩阵
print (arr)

#对角矩阵
arr = np.diag([2,2,3])
#[]中是对角线上的数字，主对角线是左上右下
print (arr)

#随机数组生成
#生成0~1之间的随机浮点数，均匀分布
arr = np.random.rand(2,3) #括号中输入形状
#试一下如果在里面再加一个小括号行不行？不行,numpy 里只有 rand 和 randn 是异类，其余全按元组传
#randn是标准正态分布
print (arr)

#生成指定区间内的浮点数
arr = np.random.uniform(2,3,(6,6))
#指的是在2~3之间生成随机浮点数，生成一个六行六列的矩阵

#生成指定区间内的整数
arr = np.random.randint(2,33,(6,6))
#指的是在2~33之间生成随机整数，生成一个六行六列的矩阵
```

补充:

类型	示例	特性
列表 list	[1, 2, 3]	内容可改、可增删
元组 tuple	(1, 2, 3)	创建后锁死、长度固定

In [ ]:
```
#正态分布
arr = np.random.randn(2,3)
#这里括号里面的是矩阵的形状，默认均值是零，标准差是一
print(arr)
```

数据类型

数据类型	说明
bool	布尔类型
int8 、 uint8	有符号、无符号的 8 位（1 字节）整型
int16 、 uint16	有符号、无符号的 16 位（2 字节）整型
int32 、 uint32	有符号、无符号的 32 位（4 字节）整型
int64 、 uint64	有符号、无符号的 64 位（8 字节）整型
float16	半精度浮点型
float32	单精度浮点型
float64	双精度浮点型
complex64	用两个 32 位浮点数表示的复数
complex128	用两个 64 位浮点数表示的复数

int8能表示数字在-2的8次方到2的8次方之间，uint 不能表示负数 精度：你的数离真值有多近，也就是"能信到第几位"。

换个说法，更好懂

In [ ]:
```
#布尔类型
arr = np.array([1,2,0],dtype=np.bool)
print(arr)#输出的是[True True False]只有0是False，其余是True
```

索引与切片

索引/切片类型	描述/用法
基本索引	通过整数索引直接访问元素。索引从 0 开始。
行/列切片	使用冒号 ： 切片语法选择行或列的子集。
连续切片	从起始索引到结束索引按步长切片。
使用 slice 函数	通过 slice(start, stop, step) 定义切片规则。
布尔索引	通过布尔条件筛选满足条件的元素。支持逻辑运算符 & 、 | 。

In [ ]:

```
# 一维数组
arr = np.random.randint(1, 100, 20)
arr = arr.reshape(4,5)
print(arr)
print("$"*100)
print(arr[2])#取一行
print("$"*100)
print(arr[:,2:])#获取第3列之后的数据
print("$"*100)
print(arr[2:4:2])#start:end+1:step
print("$"*100)
print(arr[[1,2,3]])#取不连续多行
print("$"*100)
print(arr[slice(1,2,3)])#不常用#start:end:step
print("$"*100)
print(arr[arr>10])
print("$"*100)
print(arr[(arr>10) & (arr<44)])#布尔索引：&表和，|表或
print("$"*100)
print(arr[2:5,1:4])#取行列交叉点
print("$"*100)
print(arr[[2,1],[1,2]])#取行列交叉点
```

In [ ]:

```
# 二维数组
arr = np.random.randint(1, 100, (4, 8))
print(arr)
print(arr[2,:])#从0开始数的，先行后列，：指的是全部
print(arr[2,2:3])#start:end+1:step。逗号两边是独立的行和列

print(arr[2])#过滤一个行数据
print(arr[2][arr[2] > 50])#加一个布尔

print(arr[:,3])#过滤一个列数据

print(arr[(arr>10) & (arr<44)])#整体布尔索引
```

运算

np.array的数组可以加减乘除，对应位置计算（+-*/） 但列表不能运算，只能完成列表的拼接

In [ ]:

```
#数组间运算
a = np.array([2, 3, 4, 5])
b = np.array([1, 2, 3, 4])#一维二维都一样
print(a ** b)#乘方运算也行，一一对应
print(a / b)
```

In [ ]:

```
#数组与标量运算
a = np.array([2, 3, 4, 5])
b = np.array([1, 2, 3, 4])
print(a + 3)#每一个都加3
```

In [ ]:

```
#python自带的列表
c = [1, 2, 3]
d = [2, 3, 3]
print(c + d)#只能加

#补充
for i in range(len(c)):
    d[i] = c[i] +d[i]
print(d)
```

In [ ]:

```
#广播机制（shape不一样的数组运算）
a = np.array([2, 3, 4, 5])
b = np.array([[1], [2], [3], [4]])
print(a+b)
'''
判断是否可以广播：在行或列上相同或其一为1即可
a(1*4)
2 3 4 5
b(4*1)
1
2
3
4

补全
2 3 4 5
2 3 4 5
2 3 4 5
2 3 4 5

1 1 1 1
2 2 2 2
3 3 3 3
4 4 4 4
再相加
'''
```

In [ ]:

```
#矩阵运算
a = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
b = np.array([[4, 5, 6], [7, 8, 9], [1, 2, 3]])
print(a * b)#对应相乘
print(a @ b)
#矩阵的乘法：
#结果中的（1，1）是a的第一行乘上b的第一列，即1*4+2*7+3*1y
```

补充

In [ ]:
```
import random
import numpy as np

t = np.array([random.random() for i in range(10)])#十个0到1之间的小数
t = np.round(t,2)#保留2位小数
print(t)
# 0.是指0.00
```

In [ ]:
```
#t.reshape((24,1))#仍是二维,可以只打一个括号
t1 =  t.flatten()
print(t1)
```

In [ ]:
```
import numpy as np

np.loadtxt("./data.csv",dtype=np.int,delimiter=",",skiprows=0,usecols=None,unpack=False)#csv 是逗号分隔值文件
```

参数	解释
frame	文件、字符串或产生器，可以是 .gz 或 bz2 压缩文件
dtype	数据类型，可选，CSV 的字符串以什么数据类型读入数组中，默认 np.float
delimiter	分隔字符串，默认是任何空格，改为 逗号
skiprows	跳过前x行，一般跳过第一行表头
usecols	读取指定的列，索引，元组类型
unpack	如果True，读入属性将分别写入不同数组变量，False 读入数据只写入一个数组变量，默认False

In [ ]:
```
#numpy中的转置
t.swapaxes(1,0)
t.T
t.transpose()
```

In [ ]:
```
#bool
np.where(t<10,0,10)#小于10的为0，否则为10
t.clip(10,18)#小于10换为10，大于18换为18
```

In [ ]:

```
#转nan
import numpy as np
t2 = np.arange(24).reshape(4,6)
t2 = t2.astype(np.float64)#要么float，要么np.float64/32
t2[3,3] = np.nan
print(t2)
#ValueError: cannot convert float NaN to integer  这样的报错要转成浮点数再转nan
```

In [ ]:

```
#数组拼接
np.vstack((t1,t2))
np.hstack((t1,t2))#水平拼接
#交换行列
t[[1,2],:] = t[[2,1],:]#2，3行互换
t[:,[2,3]] = t[:,[3,2]]
#读取行数
t.shape[0]
#获取最大值最小值的位置
np.argmax(t,axis=0)
np.argmin(t,axis=1)
#seed
np.random.seed(42)
#copt&view
a = b.copy()#独立存在
a = b[:]#一同变化
#
t.sum(axis=None)#求和：
t.mean(a.axis=None)#均值：受离群点的影响较大
np.median(t.axis=None)#中值：
t.max(axis=None)#最大值：
t.min(axis=None)#最小值：
np.ptp(t.axis=None)#极值：即最大值和最小值之差
t.std(axis=None)#标准差：
```

numpy-nan-mindmap (https://img.cdn1.vip/i/6ac1adc63ff00_1791077830.webp)